SIMATS ENGINEERING
Engineer to Excel

SIMATS
Saveetha Institute of Medical And Technical Sciences
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

Department of Computer Science and Engineering

DESIGN AND ANALYSIS OF ALGORITHMS

ASSIGNMENT 2 – CO2

CO2 Coverage: Brute-Force and Exhaustive-Search techniques – Selection Sort, Bubble Sort, Insertion Sort, Sequential Search, Brute-Force String Matching, Closest Pair, Convex Hull and Exhaustive Search.

Nature of Assignment: Individual
Duration: 1 Week
Total Marks: 50
Submission Date: _____

Q1 – Sorting Under Different Data Conditions 6 Marks

Problem Context – E-Commerce Order Processing

An e-commerce fulfilment centre arranges customer orders according to a **dispatch priority score**.

The development team is considering:

Selection Sort | Bubble Sort | Insertion Sort

However, the incoming data may be randomly ordered, nearly sorted, already sorted or significantly disordered.

Your Challenge

Investigate whether one of the three algorithms should always be preferred or whether the appropriate choice depends on the **characteristics of the input data**.

Before implementation:

- Predict how the three algorithms will behave under different input arrangements.
- Explain the algorithmic reason for your prediction.

Design **two deliberately contrasting datasets** and **one additional edge-case dataset of your own** that will expose meaningful behavioural differences among the algorithms.

Implement the three algorithms and record appropriate measures such as:

- Number of comparisons
- Number of swaps/movements
- Execution time, where useful

Compare your prediction with the observed behaviour.

Design Decision

The fulfilment centre reports that approximately **95% of existing orders are already in the correct priority order**, with only small batches of new orders added periodically.

Determine which of the three algorithms you would recommend and defend your decision.

Evidence Expected

Reasoned prediction, three deliberately designed input conditions, implementation of the three sorting algorithms, operation counts, comparison of predicted and observed behaviour, and an evidence-based recommendation.

Q2 – When Is Sequential Search an Appropriate Engineering Choice? 4 Marks

Problem Context – Aircraft Maintenance Tool Tracking

An aircraft maintenance facility maintains a temporary list of specialised tools currently issued to technicians.

Each record contains:

Tool ID | Technician ID | Issue Time | Work Bay

The list changes continuously as tools are issued and returned.

When a technician asks:

> “Who currently has Tool T-184?”

the system uses **Sequential Search**.

Your Challenge

Design experiments to investigate Sequential Search under:

- Successful searches at different positions
- Unsuccessful searches
- Different list sizes

Record the **number of comparisons** required.

Then answer:

> **Under what operating conditions is Sequential Search a reasonable engineering choice, and when would you stop recommending it?**

Consider both:

- Dataset size
- Frequency of search requests

Changed Condition

The organisation expands from one hangar to **20 maintenance facilities**, resulting in substantially more active records and lookup requests.

Re-evaluate your original recommendation.

Evidence Expected

Designed search experiments, comparison-count evidence, analysis of successful and unsuccessful searches, an evidence-based operating boundary and a justified response to the changed operating condition.

Q3 – Engineer the Behaviour of Brute-Force String Matching 6 Marks

Problem Context – Cybersecurity Log Screening

A security operations centre searches large text-based system logs for known event signatures represented as character patterns.

The initial system uses **Brute-Force String Matching**.

Your Challenge

Instead of merely implementing the algorithm, investigate **how the structure of the text and pattern influences computational effort**.

Design:

1. One text-pattern pair in which the algorithm performs relatively little work.
2. One text-pattern pair deliberately engineered to cause a **large number of character comparisons**.
3. One additional test condition of your own choice.

Before executing each test:

- Predict the expected behaviour.
- Explain the algorithmic reason for your prediction.

Implement the algorithm and record the number of character comparisons.

Inquiry

Compare your prediction with the experimental result.

Explain:

- Why one input requires substantially more work than another
- Which characteristics of the text and pattern create repeated comparisons
- What this reveals about evaluating an algorithm using only its input size

Evidence Expected

Correct implementation, three deliberately designed test conditions, prediction before execution, character-comparison evidence, and a reasoned explanation of the observed differences.

Q4 – Closest Pair and the Cost of Examining Every Possibility 6 Marks

Problem Context – Drone Collision-Risk Monitoring

A logistics company operates delivery drones. At a particular instant, each drone reports its position:

[
D_i=(x_i,y_i)
]

The monitoring system must identify the **two drones closest to one another**.

Your Challenge

Design a **Brute-Force Closest-Pair** solution.

Before implementation, determine:

- Which candidate pairs must be examined
- How duplicate pair comparisons can be avoided
- How many unique pairs exist for (n) drones

Derive the number of candidate pairs mathematically.

Implement your solution and test it using **at least three meaningfully different input sizes**.

Record:

- Number of pairs examined
- Number of distance calculations
- Execution time, where useful

Compare the theoretical growth with experimental behaviour.

Scalability Inquiry

Define what you consider an **acceptable response time** for your experimental environment.

Using your evidence, determine approximately when the brute-force approach begins to become impractical.

Finally, identify **where the computational inefficiency originates** and describe what a redesigned strategy would need to avoid or reduce.

You are not required to implement an advanced Closest-Pair algorithm in this question.

Evidence Expected

Algorithm design, mathematical pair-count derivation, implementation, experimental growth evidence, a defined acceptability criterion and reasoned identification of the limitation of the brute-force strategy.

Q5 – Derive a Convex Hull Algorithm from a Geometric Principle 7 Marks

Problem Context – Wildlife Reserve Boundary Monitoring

Monitoring stations are positioned throughout a wildlife reserve. Each station is represented by coordinates:

[
P_i=(x_i,y_i)
]

Conservation officers need to determine the **outer boundary formed by the monitoring stations**.

You are required to develop a **Brute-Force Convex Hull** solution.

Design Starting Point

You are given only the following geometric principle:

> **A line joining two candidate points may form part of the outer boundary if all remaining points lie on an appropriate single side of that line.**

You are **not provided with the complete algorithm**.

Your Challenge

Convert this geometric property into a computational procedure.

Determine:

- How two candidate points define a line
- How the position of another point relative to that line can be determined
- How the remaining points must be examined

- What condition identifies a potential hull edge
- How the final boundary will be constructed

Develop your own pseudocode before implementation.

Test Your Design

Test the algorithm using:

- A point set containing both boundary and interior points
- A second point configuration deliberately designed by you to challenge your algorithm

Visualise the resulting boundary.

Analyse how the computational work grows as the number of points increases.

Correctness Inquiry

Explain:

> **Why does your geometric test provide evidence that an accepted edge belongs to the outer boundary?**

Your explanation should be based on geometric reasoning rather than simply referring to the program output.

Evidence Expected

A student-derived geometric decision rule, pseudocode, implementation, deliberately designed test configurations, computational analysis, boundary visualisation and a reasoned correctness argument.

Q6 – Exhaustive Search and Combinatorial Growth 8 Marks

Problem Context – Humanitarian Airlift Planning

A helicopter carrying emergency supplies has a strict payload limit.

Each available package contains:

Package ID | Weight | Humanitarian Value

The objective is to select a combination of packages that:

- Does not exceed the payload limit, and
- Maximises the total humanitarian value delivered.

Your Challenge

Design an **Exhaustive-Search strategy**.

Determine:

- How candidate package combinations will be represented
- How candidate solutions will be generated
- How feasibility will be checked
- How feasible solutions will be evaluated
- How the best solution will be retained

Do not begin from supplied pseudocode.

Correctness Check

Create a small instance for which the possible combinations can be independently verified.

Use this to establish whether your algorithm identifies the correct optimum.

Growth Investigation

Investigate increasing problem sizes.

Before experimentation:

- Derive/predict how the number of candidate combinations grows with (n).
- Predict what will happen as additional packages are introduced.

Use **at least three meaningful problem sizes** to validate your prediction experimentally.

Inquiry

Explain:

Why can adding what appears to be a relatively small number of additional packages produce a dramatic increase in computational effort?

Your explanation must relate to the growth of the **solution space**.

Deployment Decision

Assume that the airlift planner must produce a loading decision within a fixed response time selected and justified by you.

Using your experimental evidence, estimate the largest problem size for which you would be willing to deploy your exhaustive-search solution.

Evidence Expected

Student-designed exhaustive enumeration, correctness verification, mathematical prediction of solution-space growth, experimental validation, explanation of combinatorial growth and an evidence-based deployment boundary.

DAY-4 DESIGN CHALLENGE

Q7 – Diagnose, Redesign and Validate — 13 Marks

Review the evidence you generated while working on Q1–Q6.

Select **ONE solution** that demonstrates a significant computational limitation as its problem size or operating requirement increases.

You are **not permitted simply to replace the solution with a more advanced textbook algorithm and state that its complexity is better**.

Your task is to understand the computational problem first and then develop an appropriate redesign.

A. Diagnose the Computational Bottleneck — 2 Marks

Using your own mathematical and/or experimental evidence, identify:

- Where most computational effort is being spent
- Which operation grows significantly as the input increases
- Whether work is being unnecessarily repeated
- Which characteristic of the original algorithm causes the limitation

Evidence Expected

A precise diagnosis supported by evidence from your original implementation and analysis.

B. Define the Design Objective — 2 Marks

Before proposing a new solution, clearly state what your redesign must achieve.

For example:

- Reduce unnecessary comparisons
- Avoid examining every candidate
- Exploit existing ordering
- Reuse previously computed information
- Reduce the number of candidate solutions
- Reduce repeated processing

Specify a **measurable objective** wherever possible.

Evidence Expected

A clearly stated redesign objective derived directly from the identified computational limitation.

C. Design the Revised Computational Strategy — 4 Marks

Develop a revised solution.

Explain:

- What you have changed
- What information or property your new design exploits
- Which operations are avoided or reduced
- Whether preprocessing is introduced

- Whether additional memory is required
- Any new trade-offs created by the redesign

Provide appropriate pseudocode or a design representation.

Your marks will be based on the **quality of the algorithmic reasoning**, not merely on whether you name a known algorithm.

Evidence Expected

A coherent redesigned computational strategy directly addressing the diagnosed bottleneck, supported by appropriate design representation and technical justification.

D. Predict Before Implementing — 1 Mark

Before testing the redesigned solution:

- Predict how its behaviour should differ from the original.
- Identify the performance measure in which you expect improvement.

Evidence Expected

A clear, testable prediction made before experimental validation.

E. Implement and Validate the Redesign — 2 Marks

Implement a prototype of the redesigned strategy wherever feasible.

Using the **same or equivalent test conditions**, compare:

Original Solution → Redesigned Solution

Use appropriate measures such as:

- Comparisons
- Key operations
- Candidate solutions examined
- Distance calculations
- Character comparisons
- Execution time, where relevant

Evidence Expected

Comparable experimental evidence showing the behaviour of the original and redesigned solutions.

F. Defend the Final Design — 2 Marks

Evaluate whether the redesign achieved its intended objective.

Discuss:

- Whether the prediction was supported
- Magnitude of improvement, if any
- New costs or trade-offs
- Situations in which the original solution may still be preferable
- Whether the redesigned solution sufficiently addresses the identified limitation

A redesign that does **not** outperform the original will not automatically be considered unsuccessful if you correctly analyse and explain why the design hypothesis did not produce the expected result.

Evidence Expected

A critical, evidence-based evaluation of the redesigned solution and a defensible final engineering conclusion.

MARK DISTRIBUTION

Question	Assessed Area	Marks
Q1	Selection, Bubble & Insertion Sort	**6**
Q2	Sequential Search	**4**
Q3	Brute-Force String Matching	**6**
Q4	Brute-Force Closest Pair	**6**
Q5	Brute-Force Convex Hull	**7**
Q6	Exhaustive Search	**8**

Q7	**Day-4 Design Challenge – Diagnose, Redesign & Validate**	**13**
	TOTAL	**50**

Submission Requirements

Submit:

- Your assignment report containing the required evidence for Q1–Q7
- Source code for all implemented solutions
- Raw experimental data used to produce your results
- Tables/graphs used in your analysis, where applicable

Operation counts, comparison counts or other algorithm-specific measures should be treated as **primary experimental evidence**. Execution time may be reported as additional evidence but should not be used as the sole basis for your conclusions.
